

Proyecto 2: Competencia de Agentes en Space Invaders

Un agente DQN / Double DQN / Dueling DQN para ALE/SpaceInvaders-v5

Angel Esteban Esquit Hernández
CC3092 – Deep Learning y Sistemas Inteligentes

Índice

1. Introducción	1
2. Definición del problema y análisis del entorno	1
2.1. Descripción del entorno	1
2.2. Espacio de observación y de acción	2
2.3. Análisis de la señal de recompensa	2
2.4. Preprocesamiento y decisiones de diseño	2
3. Metodología de desarrollo	3
3.1. Algoritmos considerados	3
3.2. Arquitectura de la red	3
3.3. Exploración vs. explotación	4
3.4. Hiperparámetros de entrenamiento (configuración base)	4
3.5. Protocolo de entrenamiento y evaluación	4
3.6. Baselines de referencia	5
4. Primera ronda de iteraciones y resultados	5
5. Limitación del protocolo de evaluación de 5 episodios	6
6. Búsqueda extendida: más semillas y entrenamiento de largo plazo	7
6.1. iter05_sticky5m (semillas 0, 1, 2) – entrenamiento con sticky actions	7
6.2. iter06_det5m (semillas 3, 4) – réplicas deterministas	7
6.3. iter07_det8m (semilla 5, Google Colab/Kaggle) – 8M pasos	7
6.4. iter08_detlong_s6 – corrida larga (objetivo 100M pasos)	7
7. Decisión final del agente de competencia	8
8. Evaluación cualitativa (video)	9

9. Discusión general	9
10. Limitaciones	10
11. Trabajo futuro	11
12. Conclusiones	11
13. Repositorio de GitHub	12
Referencias	12

1. Introducción

El objetivo de este proyecto es construir un agente capaz de jugar *Space Invaders* (Atari 2600, vía Arcade Learning Environment/Gymnasium) mejor que dos baselines de referencia sin aprendizaje, usando técnicas de *Deep Q-Learning*. El proyecto se construye sobre las utilidades ya desarrolladas en el Laboratorio #5 (`ale_utils.py: crear_entorno, ejecutar_episodio, generar_video_agente`), reemplazando el agente aleatorio o de regla simple por una política aprendida mediante **DQN**, con las extensiones **Double DQN** y **Dueling DQN** activables por configuración.

El desarrollo se documenta íntegramente en el repositorio de GitHub del proyecto (ver Sección 13), incluyendo el código fuente, los registros de entrenamiento (`logs/`), los checkpoints de los mejores agentes (`checkpoints/`) y los videos de evaluación (`videos/`). Este documento resume ese trabajo: el análisis del entorno, las decisiones de diseño, las iteraciones de entrenamiento realizadas, un problema metodológico que se detectó a mitad del proyecto en el protocolo de evaluación, la búsqueda extendida que se hizo para resolverlo, y la decisión final del agente presentado en la competencia.

2. Definición del problema y análisis del entorno

2.1. Descripción del entorno

ALE/SpaceInvaders-v5 simula el juego de Atari 2600 *Space Invaders*: el jugador controla una nave que se mueve horizontalmente en la parte inferior de la pantalla y dispara contra hileras de invasores alienígenas que descienden progresivamente. El objetivo es destruir la mayor cantidad de invasores posible antes de que estos lleguen a la base del jugador o de perder las 3 vidas disponibles.

- **Recompensa:** puntos positivos al destruir un invasor (varía según la fila/tipo de invasor: los de filas superiores valen más puntos), sin penalización explícita por perder una vida o por disparar.
- **Fin de episodio:** `terminated=True` cuando el jugador pierde sus 3 vidas (game over) o los invasores llegan a la base; `truncated=True` si se alcanza el límite de pasos del entorno (`TimeLimit`, grande por defecto para Atari).
- **Vidas:** 3 vidas por partida (consultable vía `info["lives"]` o `env.unwrapped.ale.lives()`).

2.2. Espacio de observación y de acción

Observación cruda: imagen RGB de (210,160,3) píxeles por defecto (`obs_type="rgb"`). ALE también permite `obs_type="grayscale"` o `obs_type="ram"` (128 bytes de la RAM del emulador).

Espacio de acción mínimo (`full_action_space=False`, usado en este proyecto): `Discrete(6)` – NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE, LEFTFIRE. El espacio de acción completo de ALE (`full_action_space=True`) tiene 18 acciones, muchas sin efecto en este juego (p. ej. UP, DOWN).

Decisión de diseño: se usa el subconjunto mínimo de 6 acciones. Ampliar a las 18 acciones no agrega expresividad útil (el jugador solo se mueve horizontalmente) y sí incrementa el espacio de exploración que la red debe aprender a descartar, ralentizando el entrenamiento.

2.3. Análisis de la señal de recompensa

Para caracterizar qué tan densa o dispersa es la recompensa se ejecutó un episodio con el agente aleatorio del Laboratorio #5, midiendo en qué fracción de los pasos se recibe recompensa distinta de cero. La recompensa es **dispersa**: la gran mayoría de los pasos (moverse, disparar sin acertar) reciben recompensa 0, y solo una fracción pequeña de pasos (impactar un invasor) produce recompensa positiva. Además, la magnitud de la recompensa varía según qué invasor se destruye (entre 5 y 30 puntos aproximadamente, según la fila), lo que motiva **recortar (clip) la recompensa a $\{-1, 0, 1\}$** durante el entrenamiento (como en Mnih et al., 2015), para que la magnitud del error de TD sea comparable entre pasos y estabilizar el entrenamiento. En evaluación y en la competencia se reporta la recompensa real sin recortar, como exige el enunciado.

No se observó necesidad de *reward shaping* adicional: la señal original, aunque dispersa, es suficientemente frecuente en Space Invaders (hay muchos invasores que destruir) para que el replay buffer acumule transiciones informativas sin ayuda externa.

2.4. Preprocesamiento y decisiones de diseño

Implementado en `dqn/wrappers.py::crear_entorno_dqn`, construido *encima* de `ale_utils.crea_r_entorno` (sin reemplazarlo):

Paso	Wrapper	Motivo
Escala de grises + resize 84×84	AtariPreprocessing	El color no es relevante para identificar naves/invasores/proyectiles; reduce drásticamente el tamaño de entrada de la CNN.
Frame-skip = 4 con max-pool de 2 frames	AtariPreprocessing	Los frames consecutivos son casi idénticos y algunos sprites parpadean cada frame en el hardware original de Atari; el max-pool evita perderlos. Repetir la acción 4 pasos también acelera el entrenamiento.
Apilado de 4 frames	FrameStackObservation	Una sola imagen no revela dirección/velocidad de balas ni invasores (entorno parcialmente observable con un solo frame); apilar 4 frames se lo da al agente sin necesidad de una red recurrente.
NOOP aleatorio al inicio (hasta 30)	AtariPreprocessing	Evita que el agente memorice la secuencia exacta de frames iniciales de cada episodio.
Fin de episodio por vida perdida	EpisodicLifeWrapper (propio)	Solo en entrenamiento: al perder una vida se reporta <code>terminated=True</code> para propagar antes la señal negativa, sin reiniciar la partida real.
Reward clipping a {-1, 0, 1}	ClipReward	Solo en entrenamiento (ver sección anterior).

3. Metodología de desarrollo

3.1. Algoritmos considerados

Se implementó **DQN** (Mnih et al., 2015) desde cero en PyTorch (`dqn/agent.py`), con soporte para dos extensiones activables por configuración:

- **Double DQN** (Van Hasselt et al., 2016): la red *online* elige la mejor acción del siguiente estado y la red *objetivo* solo la evalúa, reduciendo la sobreestimación sistemática de los valores Q que sufre el DQN original (que usa la misma red objetivo para elegir y evaluar).
- **Dueling DQN** (Wang et al., 2016): arquitectura alternativa que separa la estimación de $V(s)$ y $A(s, a)$, combinadas como $Q(s, a) = V(s) + (A(s, a) - \text{mean}_{a'} A(s, a'))$.

Se descartó implementar métodos *on-policy* (PPO/A2C) porque DQN y sus variantes son más simples de depurar y comparar entre sí manteniendo la misma arquitectura convolucional y el mismo replay buffer, lo que facilita aislar el efecto de cada cambio (arquitectura vs. algoritmo de target) en la sección de resultados.

3.2. Arquitectura de la red

NatureCNN (`dqn/model.py`), la arquitectura convolucional original de Mnih et al. (2015):

Capa	Detalle	Salida
Entrada	4 frames apilados, 84×84 , <code>uint8</code> normalizado a $[0, 1]$	(4, 84, 84)
Conv1	32 filtros, kernel 8×8 , stride 4, ReLU	(32, 20, 20)
Conv2	64 filtros, kernel 4×4 , stride 2, ReLU	(64, 9, 9)
Conv3	64 filtros, kernel 3×3 , stride 1, ReLU	(64, 7, 7)
Flatten + FC	512 unidades, ReLU	(512,)
Salida	<code>n_actions</code> valores Q (o $V(s) + A(s, a)$ en Dueling)	(6,)

3.3. Exploración vs. explotación

Epsilon-greedy con decaimiento **lineal** de $\epsilon = 1.0$ a $\epsilon = 0.02$ (en la mayoría de las corridas finales) a lo largo de `epsilon_decay_steps` pasos de entorno (1,000,000 en todas las corridas reportadas). Durante evaluación se usa política puramente greedy ($\epsilon = 0$), como exige el protocolo de competencia.

3.4. Hiperparámetros de entrenamiento (configuración base)

Hiperparámetro	Valor	Notas
Función de pérdida	Huber / Smooth L1	Menos sensible a outliers en el error de TD que MSE
Optimizador	Adam	<code>lr</code> = $2.5e-4$
Factor de descuento γ	0.99	
Tamaño del replay buffer	150,000 – 200,000 transiciones	Limitado por RAM disponible
Tamaño de batch	32	
Pasos de calentamiento	50,000	Pasos aleatorios antes de entrenar, para llenar el buffer
Frecuencia de entrenamiento	cada 4 pasos de entorno	
Frecuencia de actualización de la red objetivo	cada 10,000 pasos de gradiente	

3.5. Protocolo de entrenamiento y evaluación

El entrenamiento se ejecuta desde línea de comandos (`python -m dqn.train`), no desde el notebook, porque toma desde decenas de minutos hasta decenas de horas según el número de pasos. Cada corrida registra una fila por episodio en `logs/<iteracion>.csv` (recompensa, longitud, epsilon y pérdida promedio) y guarda checkpoints periódicos en `checkpoints/<iteracion>.pt`.

La evaluación de competencia (`evaluate.py`) reconstruye la arquitectura desde la configuración guardada en el checkpoint, ejecuta 5 episodios completos con política **greedy** sobre el entorno real (recompensa sin recortar, episodio = partida completa de 3 vidas, sin cortar por vida perdida), imprime promedio y máximo, y graba un video `.mp4` por episodio.

3.6. Baselines de referencia

Se compara contra los dos agentes sin aprendizaje ya implementados en el Laboratorio #5, evaluados con el mismo protocolo (5 episodios, recompensa real sin recortar):

- **Agente aleatorio:** acción aleatoria en cada paso (baseline estándar en RL, equivalente a “predecir la media” en un problema de regresión).
- **Agente de regla simple:** política fija (dispara siempre que puede), sin ningún tipo de aprendizaje.

4. Primera ronda de iteraciones y resultados

Se realizaron nueve corridas de entrenamiento para aislar el efecto de Double DQN, Dueling DQN y el número de pasos de entrenamiento, evaluadas inicialmente con el protocolo de competencia de 5 episodios greedy.

Iteración	Cambios vs. best_run	Pasos	Reward prom.	Reward máx.
Baseline aleatorio	– (sin aprendizaje)	–	185.0	260.0
Baseline regla simple	– (sin aprendizaje)	–	270.0	270.0
best_run	Dueling + Double DQN, $\gamma=0.99$, $lr=2.5e-4$, buffer=150k, ϵ : 1.0→0.02 en 1M pasos	3,000,000	710.0	800.0
iter03	Igual a best_run , continuado con <code>-resume-from</code> desde el checkpoint de 3M	5M (+2M)	634.0	655.0
iter_plain_dqn	Sin Double DQN ni Dueling (DQN vanilla)	1,500,000	602.0	755.0
iter_double_only	Solo Double DQN	1,500,000	385.0	490.0
iter_dueling_onl y	Solo Dueling	1,500,000	360.0	460.0
iter04_clean5m	Igual a best_run , entrenado desde cero hasta 5M pasos	5,000,000	703.0	1300.0
iter_double_only_3m	Solo Double DQN, desde best_run desde cero, mismos pasos que	3,000,000	782.0	900.0
iter_dueling_onl y_3m	Solo Dueling, desde cero, mismos pasos que best_run	3,000,000	582.0	750.0

Ninguna divergencia de los valores Q se observó en ninguna corrida: la pérdida se mantuvo estable y baja (~ 0.01 – 0.02) durante todo el entrenamiento en todos los casos.

Hallazgo metodológico (no de estabilidad): reanudar entrenamiento desde un checkpoint con ϵ ya decaído (**iter03**) empeora el resultado (710.0→634.0 de promedio), porque el replay buffer **no se persiste** entre corridas y se rellena con transiciones poco diversas (política casi greedy) en vez de la mezcla exploratoria que tuvo el buffer original durante el decaimiento de ϵ . Entrenar más

pasos **desde cero** (`iter04_clean5m`) no sufre este problema: su promedio (703.0) queda a la par de `best_run` (710.0).

Ablaciones a 1.5M pasos, engañosas: a esa cantidad de pasos ninguna configuración individual (`iter_plain_dqn`, `iter_double_only`, `iter_dueling_only`) había convergido todavía, y las tres quedaron por debajo de lo esperado, sin un patrón claro. Al repetir la ablación a 3M pasos (`iter_double_only_3m`, `iter_dueling_only_3m`), el panorama cambió por completo: **Double DQN por sí solo explica la mayor parte de la ganancia** de `best_run` sobre las variantes sin mejoras (782.0/900.0, incluso por encima de `best_run`), mientras que Dueling por sí solo ayuda menos y converge más lento (582.0/750.0). Esta es una lección clara sobre no sacar conclusiones de ablaciones antes de verificar convergencia aproximada.

5. Limitación del protocolo de evaluación de 5 episodios

El protocolo de competencia evalúa con 5 episodios greedy. Sin embargo, con $\epsilon = 0$ el entorno de entrenamiento (`repeat_action_probability=0.0`, determinista salvo por el NOOP inicial aleatorio) produce muy pocas trayectorias distintas: en la práctica, solo 2–4 puntajes distintos en 5 episodios, por lo que el promedio y el máximo reportados son **muy ruidosos** y pueden depender más de la semilla que de la calidad real de la política.

Para cuantificar este problema se construyó `evaluate_robustness.py`, que evalúa cada checkpoint con un número mayor de episodios (50–200) bajo **dos configuraciones** del entorno:

- **Determinista** (`repeat_action_probability = 0.0`): la usada en entrenamiento y en `evaluate.py`.
- **Sticky actions** (`repeat_action_probability = 0.25`): el valor por defecto de ALE/Space Invaders-v5 en Gymnasium, que introduce estocasticidad repitiendo la acción anterior con probabilidad 0.25 en vez de ejecutar la acción elegida.

Al re-evaluar los mejores candidatos de la Sección 4 con 50 episodios en ambas configuraciones (`logs/robustness_eval.csv`), el orden cambió sustancialmente respecto al reportado con 5 episodios:

Checkpoint	Det. media	Det. máx.	Sticky media	Sticky máx.
<code>best_run</code>	721.2	800.0	524.2	950.0
<code>iter04_clean5m</code>	726.0	1300.0	764.9	1345.0
<code>iter_double_only_3m</code>	656.2	900.0	563.4	1230.0
<code>iter_dueling_only_3m</code>	603.2	750.0	590.1	880.0
<code>iter03</code>	606.6	655.0	671.9	1630.0

Con 50 episodios, `iter04_clean5m` deja de verse claramente por debajo de `iter_double_only_3m` (que ganaba con el protocolo de 5 episodios) y en la configuración sticky incluso lo supera. Esto confirmó que la elección del agente final **no podía basarse en 5 episodios** y motivó una búsqueda extendida: más semillas por configuración y una corrida de entrenamiento mucho más larga, evaluadas todas con este protocolo de robustez.

6. Búsqueda extendida: más semillas y entrenamiento de largo plazo

Sobre la configuración de `best_run / iter04_clean5m` (Dueling + Double DQN, $\gamma=0.99$, $\text{lr}=2.5\text{e-}4$, $\text{buffer}=150\text{k}$, $\epsilon: 1.0 \rightarrow 0.02$ en 1M pasos) se lanzaron varias colas de entrenamiento desatendidas (`run_queue.sh` a `run_queue4.sh`, `run_watch_extra.sh`) para:

1. Probar el efecto de entrenar directamente con *sticky actions* (`-sticky-prob 0.25`) en vez de solo evaluarlas.
2. Repetir la configuración determinista con semillas adicionales.
3. Entrenar una corrida mucho más larga con checkpoints (*snapshots*) ligeros periódicos, para ver si más pasos seguían mejorando el agente cuando se entrena desde cero (a diferencia del artefacto de `iter03`).

6.1. `iter05_sticky5m` (semillas 0, 1, 2) – entrenamiento con sticky actions

Tres semillas de 5M pasos entrenadas con `-sticky-prob 0.25`. Resultado mixto: las semillas 0 y 2 alcanzan medias de robustez (50 episodios) en el rango 500–700, similares a las corridas deterministas, pero la **semilla 1 colapsó a una política degenerada**: los 5 snapshots evaluados (de 1M a 5M pasos) dieron exactamente 270.0 de recompensa en el 100 % de los 50 episodios, en ambas configuraciones – es decir, una política completamente determinista y estancada (probablemente equivalente a la del baseline de regla simple), sin ninguna variación entre episodios. Esto ilustra que entrenar con sticky actions no garantiza una política más robusta y puede, con cierta semilla, colapsar a un óptimo local pobre.

6.2. `iter06_det5m` (semillas 3, 4) – réplicas deterministas

Repetición de la configuración determinista de `iter04_clean5m` con nuevas semillas. La semilla 3, evaluada con 200 episodios, dio 775.35/980.0 (determinista) y 565.0/1375.0 (sticky) – consistente con el rango de las corridas anteriores, sin mejoras claras.

6.3. `iter07_det8m` (semilla 5, Google Colab/Kaggle) – 8M pasos

Corrida ejecutada en Kaggle (GPU) hasta 8M pasos, evaluada por snapshots (4M a 8M pasos, 100 episodios cada uno). Los resultados oscilan en el rango 547–715 (determinista) y 600–697 (sticky), sin una tendencia ascendente clara a esta escala de pasos; no resultó competitiva frente a las mejores corridas.

6.4. `iter08_detlong_s6` – corrida larga (objetivo 100M pasos)

La corrida más ambiciosa: misma configuración que `iter04_clean5m`, semilla 6, con `-snapshot-freq 1000000 -snapshot-light` para guardar un checkpoint ligero cada millón de pasos sin detener el entrenamiento. Se lanzó con un objetivo de 100M pasos y se dejó corriendo de forma desatendida mientras un proceso vigilante evaluaba cada snapshot nuevo con 100 episodios (semillas desde 5000, ambas configuraciones) en `logs/robustness_long/`. Tras aproximadamente 45 horas de entrenamiento continuo ($\sim 1.23\text{M}$ pasos/hora) el proceso alcanzó los 55.5M pasos sin señales de divergencia

(pérdida estable, recompensa de entrenamiento de las últimas 200 corridas ~ 16.3 recortada), momento en el que se congeló la decisión para la entrega.

Se evaluaron snapshots desde los 5M hasta los 57M pasos (`logs/robustness_status.md` / `decision_status.md` recoge la tabla completa). El puntaje combinado (promedio de la media sticky y la media determinista) fluctúa considerablemente de un millón de pasos al siguiente – consistente con que una sola semilla de entrenamiento no converge monótonamente – pero destaca una **ventana clara alrededor de los pasos 40M–42M**, con puntajes de 881, 921 y 990 respectivamente (los tres marcados como candidatos, “top3”), muy por encima del resto de snapshots (típicamente 550–750).

7. Decisión final del agente de competencia

Para elegir el checkpoint final se aplicó un protocolo de confirmación en dos pasos: (1) identificar los mejores candidatos por su puntaje con 100 episodios, y (2) volver a evaluarlos con **200 episodios frescos** (semillas nuevas, desde 9000) para confirmar que la ventaja no era ruido de evaluación. Se definió una regla explícita: solo se reemplaza al mejor candidato anterior (`iter04_clean5m`, el mejor de la primera ronda con evaluación robusta) si el puntaje confirmado de un retador es al menos 5 % mayor y su máximo bajo sticky actions es al menos 1200.

Checkpoint	Determinista (media/mediana/máx.)	Sticky (media/mediana/máx.)	Puntaje	vs. iter04
42M (elegido)	1175 / 1375 / 1495	738 / 600 / 1860	956.3	+26.2 %
41M	1204 / 1230 / 1500	707 / 600 / 1745	955.2	+26.0 %
40M	972 / 755 / 1830	713 / 600 / 1955	842.4	+11.1 %
51M	912 / 1065 / 1205	769 / 600 / 2060	840.5	+10.9 %
<code>iter04_clean5m</code> (fallback)	765 / 665 / 1300	751 / 600 / 1815	758.0	–
53M	758 / 575 / 1085	692 / 583 / 1835	724.7	-4.4 %
50M	581 / 600 / 725	645 / 605 / 1210	613.2	-19.1 %

Checkpoint elegido: `checkpoints/iter08_detlong_s6_42000k.pt` (snapshot a 42M pasos de la corrida larga `iter08_detlong_s6`; Dueling + Double DQN, entrenado **sin** sticky actions). **Checkpoint de respaldo:** `checkpoints/iter04_clean5m.pt` (5M pasos, misma configuración base). Ambos superan la regla de reemplazo (42M: +26.2 % de puntaje y máximo sticky de $1860 \geq 1200$); 41M queda esencialmente empatado con 42M (955.2 vs. 956.3), por lo que 42M se mantiene como elección por tener el puntaje ligeramente más alto en la confirmación.

Comando de evaluación usado para la presentación:

```
python evaluate.py --checkpoint checkpoints/iter08_detlong_s6_42000k.pt \
  --episodes 5 --video-folder videos
```

Con el protocolo oficial de 5 episodios (semillas por defecto de `evaluate.py`) el resultado fue [1115, 775, 1495, 775, 775], media 987.0, máximo 1495.0 – claramente por encima de todas las iteraciones de la primera ronda (Sección 4).

Advertencias sobre la elección

- **Toda la ganancia proviene de la configuración determinista.** Bajo sticky actions (0.25) el snapshot de 42M queda prácticamente empatado con `iter04_clean5m` (738 vs. 751 de media;

1860 vs. 1815 de máximo). Si el entorno de evaluación del curso usa sticky actions, se espera un resultado similar al del checkpoint de respaldo; si es determinista (como el protocolo oficial descrito en el enunciado), se espera un resultado claramente mejor (mediana 1375 vs. 665).

- En configuración determinista solo hay ~ 4 trayectorias distintas por cada 200 episodios, por lo que esa confirmación es menos informativa de lo que sugiere el número de episodios. El resultado a 42M (1175 de media) es consistente con una evaluación previa de 100 episodios (1203), lo cual da cierta confianza, aunque las muestras siguen siendo pocas trayectorias independientes.
- Los snapshots de 40M, 41M y 42M son consecutivos dentro de un mismo tramo de **una sola** corrida de entrenamiento. Los snapshots evaluados entre 43M y 50M volvieron a caer a un puntaje de ~ 585 – 750 , por lo que 42M parece ser un buen momento de una política que fluctúa, no un nivel permanentemente superior. Aun así, es el mejor medido con el protocolo de confirmación.
- Los checkpoints `.pt` normalmente se excluyen del repositorio por tamaño (`.gitignore`); para esta entrega se versionaron explícitamente los 5 mejores candidatos confirmados (`iter04_clean5m`, y los snapshots 40M/41M/42M/51M de `iter08_detlong_s6`).

8. Evaluación cualitativa (video)

El video de evaluación del agente final (`videos/iter08_detlong_s6_42000k/`, 5 episodios) muestra un comportamiento consistente con el de las mejores corridas anteriores (`best_run / iter_double_only_3m`): el agente evita quedarse quieto o pegado a un extremo de la pantalla, se desplaza lateralmente disparando de forma casi continua, y prioriza despejar las columnas de invasores más cercanas a su posición actual en vez de perseguir invasores de mayor puntaje en filas superiores más lejanas – consistente con una política que maximiza recompensa esperada a corto plazo dado el descuento $\gamma = 0.99$. No se observa un comportamiento claro de esquivar proyectiles enemigos específicos (el movimiento horizontal parece más reactivo a la posición de los invasores que a las balas), lo que se refleja en la varianza entre episodios de evaluación: partidas donde el agente recibe pocos disparos tempranos avanzan mucho más que aquellas donde pierde una vida rápido.

9. Discusión general

Qué cambio tuvo mayor impacto en el puntaje. De las corridas de la primera ronda, el factor individual con mayor impacto fue **Double DQN**: a 3M pasos, `iter_double_only_3m` superó tanto a `best_run` (que combina Double+Dueling) como, claramente, a `iter_dueling_only_3m` (solo Dueling). Dueling ayuda, pero aporta menos y converge más lento. El número de pasos de entrenamiento fue el segundo factor más relevante: ninguna ablación a 1.5M pasos había convergido todavía, lo que subestimó el efecto real de cada mejora en esa primera comparación.

Continuar entrenando no es una mejora gratuita si se reanuda desde un checkpoint. `iter03` (resumir `best_run` hasta 5M pasos) empeoró el resultado, mientras que `iter04_clean5m` (entrenar 5M pasos desde cero con los mismos hiperparámetros) se mantuvo a la par. La diferencia no está en la cantidad de pasos, sino en que el replay buffer **no se persiste** entre corridas: reanudar con ϵ ya en 0.02 llena el buffer con transiciones poco diversas (política casi greedy), mientras que entrenar desde cero atraviesa la fase completa de decaimiento de ϵ ($1.0 \rightarrow 0.02$), que genera un buffer

más exploratorio. Es una limitación concreta de la implementación actual (no se guarda el estado del replay buffer en el checkpoint), no del algoritmo DQN en sí. La corrida larga `iter08_detlong_s6` evita este problema por construcción, ya que nunca se reanuda: entrena desde cero de forma continua hasta 55.5M pasos.

El protocolo de evaluación de 5 episodios es insuficiente para elegir entre candidatos cercanos. Como se documenta en la Sección 5, el entorno determinista produce muy pocas trayectorias distintas en 5 episodios, y el orden de los candidatos cambió al evaluar con 50–200 episodios. Este fue el hallazgo metodológico más importante del proyecto: motivó construir `evaluate_robustness.py` y rediseñar todo el proceso de selección del agente final en torno a evaluaciones más grandes y una regla de reemplazo explícita (Sección 7), en vez de simplemente reportar el número más alto obtenido en algún momento del entrenamiento.

Entrenar con sticky actions no garantiza mayor robustez. La semilla 1 de `iter05_sticky5m` colapsó a una política degenerada y completamente determinista (270.0 de recompensa en el 100 % de los episodios evaluados), a pesar de entrenarse con estocasticidad en el entorno. Esto sugiere que la varianza entre semillas puede dominar sobre el efecto de entrenar con o sin sticky actions, al menos con los hiperparámetros e infraestructura usados (sin repetir cada configuración con múltiples semillas de forma sistemática).

Trade-off exploración/explotación. Durante la fase de ϵ alto (~ 1.0 a 0.1 , primeros $\sim 800k$ – $900k$ pasos) la recompensa de entrenamiento es baja y ruidosa; a medida que ϵ decae hacia 0.02 la recompensa de entrenamiento sube y se estabiliza. No se observó el problema opuesto (ϵ decayendo demasiado rápido y estancando al agente en una política subóptima) con `epsilon_decay_steps = 1,000,000`; el problema real encontrado fue el ya mencionado del buffer al reanudar entrenamiento, que es un caso distinto.

10. Limitaciones

- Todas las conclusiones se basan, en su mayoría, en **una sola semilla por configuración** en la primera ronda de ablaciones; la búsqueda extendida (Sección 6) agregó algunas semillas adicionales, pero no de forma sistemática para todas las combinaciones de Double DQN/Dueling. Parte de la variación observada entre corridas (p. ej. saltos puntuales como el máximo de 1495–1955 en algunos snapshots) podría explicarse por varianza de entrenamiento más que por un efecto sistemático de los hiperparámetros.
- El cómputo disponible (una GPU local NVIDIA GTX 1660 Super y sesiones de Google Colab/Kaggle con GPU T4) limitó el número total de corridas y su duración máxima práctica; la corrida más larga alcanzó 55.5M pasos, mientras que la literatura suele entrenar decenas de millones de pasos por corrida para resultados fuertemente convergidos (p. ej. Mnih et al., 2015 entrena $\sim 50M$ pasos por juego, con múltiples semillas).
- El replay buffer no se persiste en el checkpoint, por lo que `-resume-from` no es equivalente a “entrenar esos mismos pasos desde el inicio” una vez que ϵ ya decayó (ver `iter03`).
- La confirmación de 200 episodios en configuración determinista sigue reflejando solo un puñado de trayectorias distintas del entorno, por lo que su poder estadístico real es menor al que sugiere el número de episodios.

11. Trabajo futuro

1. Repetir las configuraciones clave con múltiples semillas de forma sistemática, para separar el efecto sistemático de cada mejora de la varianza de entrenamiento.
2. Persistir el replay buffer en el checkpoint, para que `-resume-from` sea un verdadero “continuar” en vez de reiniciar el buffer.
3. Probar *Prioritized Experience Replay*, que debería beneficiar más a la variante sin Dueling al priorizar transiciones de alto error de TD.
4. Completar la ablación controlada por número de pasos entrenando también `iter_plain_dqn` (sin ninguna mejora) a 3M–5M pasos.
5. Dado que la ventana de buen desempeño en `iter08_detlong_s6` apareció alrededor de los 40M–42M pasos y luego volvió a bajar, investigar si esa fluctuación es un patrón recurrente (p. ej. relacionado con la actualización periódica de la red objetivo o con el vaciado/renovación del replay buffer) que se pueda anticipar o evitar, en vez de tratarlo como una ventana afortunada.

12. Conclusiones

- **Desempeño final:** el agente elegido para la competencia es el snapshot a 42M pasos de la corrida `iter08_detlong_s6` (Dueling + Double DQN, entrenado desde cero, sin sticky actions), con 987.0 de recompensa promedio y 1495.0 de recompensa máxima en los 5 episodios del protocolo oficial – claramente por encima de todas las iteraciones previas y de ambos baselines (185.0/260.0 el aleatorio, 270.0/270.0 el de regla simple), confirmando que el agente aprendió una política no trivial de Space Invaders.
- **Double DQN aportó más que Dueling** en este entorno, al menos con los hiperparámetros y el número de pasos probados en la ablación controlada a 3M pasos.
- **El protocolo de evaluación importa tanto como el algoritmo:** evaluar con 5 episodios en un entorno casi determinista puede invertir el orden de candidatos cercanos; ampliar a decenas o cientos de episodios, en dos configuraciones del entorno, fue necesario para tomar una decisión final defendible.
- **Entrenar más tiempo ayuda, pero no de forma monótona ni gratuita:** reanudar entrenamiento desde un checkpoint con ϵ ya decaído perjudicó al agente (`iter03`), mientras que entrenar desde cero por más tiempo (`iter04_clean5m`, `iter08_detlong_s6`) sí permitió seguir mejorando, aunque con fluctuaciones importantes de un millón de pasos al siguiente.
- **Llevar un registro de iteraciones y resultados durante el desarrollo** (en vez de reconstruirlo al final) permitió detectar el problema de `iter03`, diseñar `iter04_clean5m` e `iter_double_only_3m/iter_dueling_only_3m` específicamente para aislarlo, y finalmente identificar y corregir la limitación del protocolo de evaluación de 5 episodios, en vez de simplemente reportar el número más alto obtenido en cualquier momento del entrenamiento.

13. Repositorio de GitHub

<https://github.com/AngelEsquit/PRY2-DL>

Ver `README.md` del repositorio para instrucciones detalladas de instalación, reproducción del entrenamiento y evaluación/carga del modelo final. El análisis exploratorio del entorno y las gráficas de curvas de entrenamiento se encuentran en `notebooks/Proyecto2_SpaceInvaders_DQN.ipynb`.

Referencias

- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). *Human-level control through deep reinforcement learning*. *Nature*, 518(7540), 529–533.
- Van Hasselt, H., Guez, A., & Silver, D. (2016). *Deep Reinforcement Learning with Double Q-learning*. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1).
- Wang, Z., Schaul, T., Hessel, M., et al. (2016). *Dueling Network Architectures for Deep Reinforcement Learning*. *Proceedings of the 33rd International Conference on Machine Learning*.